Line Style Line style is used to identify the individual functions when plotting multiple functions in a function plot.

Each individual mathematical function can be assigned a special line style.

How can we visually distinguish several mathematical functions plotted on the same graph?

Let us discuss this through a problem.

Problem A mathematical signal is composed of a basic sinusoidal oscillation and its higher harmonics. Write a Python program using NumPy and Matplotlib to:

1. Generate 500 equally spaced values of x from approximately 0 to 2pi.
2. Calculate the following sinusoidal functions:

   y1=sin(x)

   y3=sin(3*x)/3

   y5=sin(5x)/5

   y7=sin(7x)/7
3. Calculate the combined function:

   y=y1+y3+y5+y7
4. Plot all five functions on the same set of axes.
5. Use different line styles and colors to distinguish the individual functions.
6. Label the x-axis and y-axis and display grid lines.

The purpose is to demonstrate how line styles can be used to distinguish multiple curves in a single plot.

Suppose we plot five curves together. If all curves have the same color and same solid line, it becomes difficult to identify which curve represents which mathematical function.

1. plot(x,y1)

```
import numpy as np
import matplotlib.pyplot as plt

x=np.linspace(0,6.3,500)

y1=np.sin(x)
y3=np.sin(3*x)/3
```

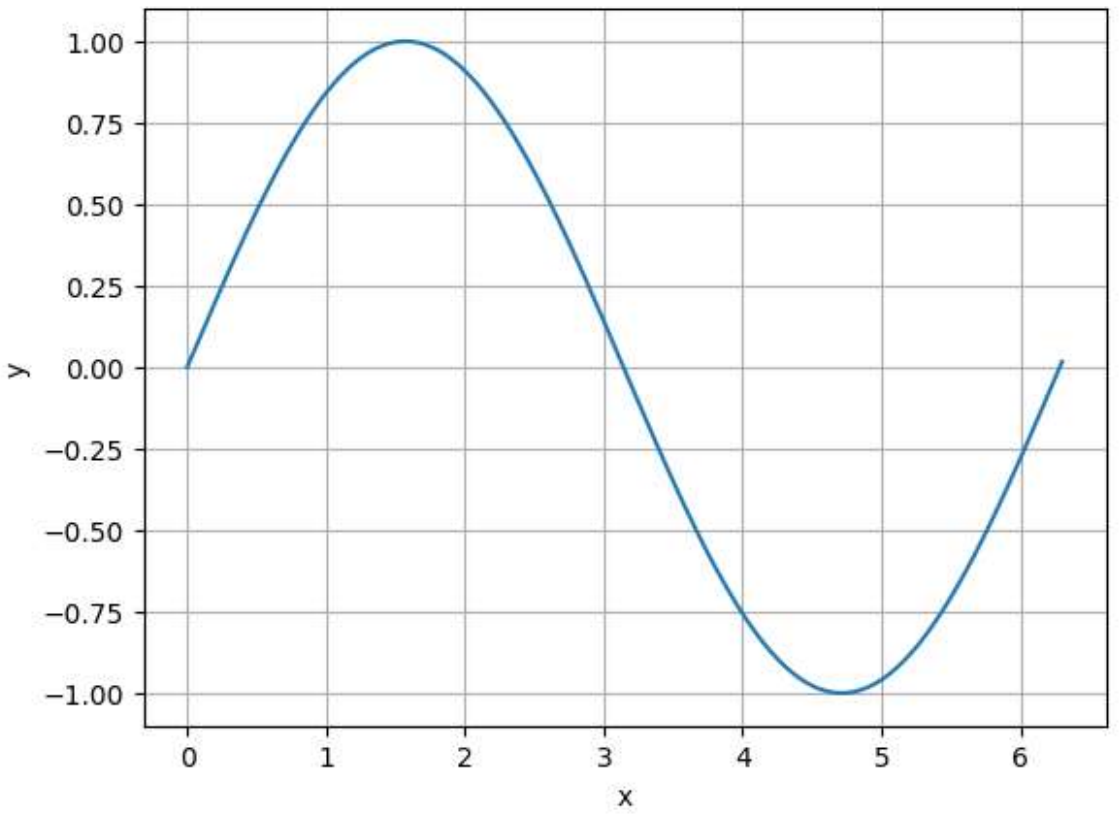
```
y5=np.sin(5*x)/5
y7=np.sin(7*x)/7

y=y1+y3+y5+y7

fig,ax=plt.subplots()
ax.plot(x,y1)
#ax.plot(x,y3)
#ax.plot(x,y5)
#ax.plot(x,y7)
#ax.plot(x,y)

ax.set_xlabel('x')
ax.set_ylabel('y')
ax.grid(True)

plt.show()
```



2. plot vothe x,y1 and x,y2

```
import numpy as np
import matplotlib.pyplot as plt

x=np.linspace(0,6.3,500)

y1=np.sin(x)
y3=np.sin(3*x)/3
```

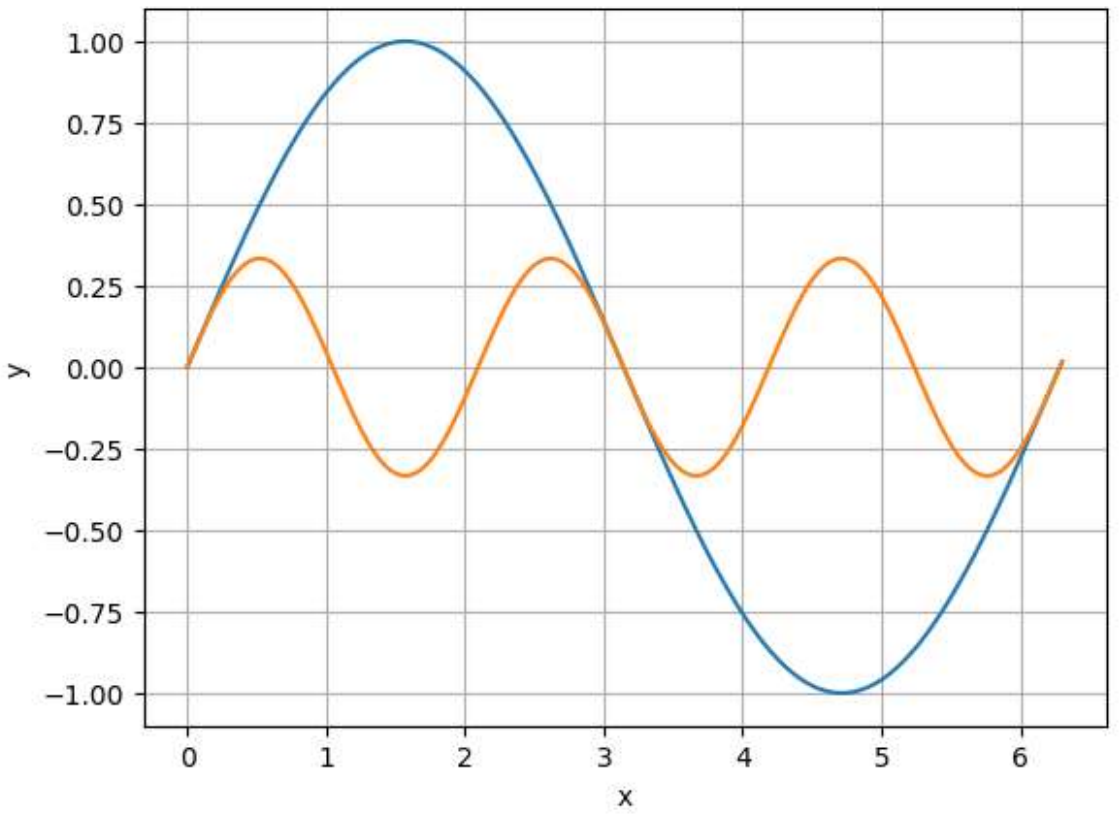
```
y5=np.sin(5*x)/5
y7=np.sin(7*x)/7

y=y1+y3+y5+y7

fig,ax=plt.subplots()
ax.plot(x,y1)
ax.plot(x,y3)
#ax.plot(x,y5)
#ax.plot(x,y7)
#ax.plot(x,y)

ax.set_xlabel('x')
ax.set_ylabel('y')
ax.grid(True)

plt.show()
```



3. plot(x,y1),Plot(x,y3) and plot(x,y5)

```
import numpy as np
import matplotlib.pyplot as plt

x=np.linspace(0,6.3,500)

y1=np.sin(x)
y3=np.sin(3*x)/3
```

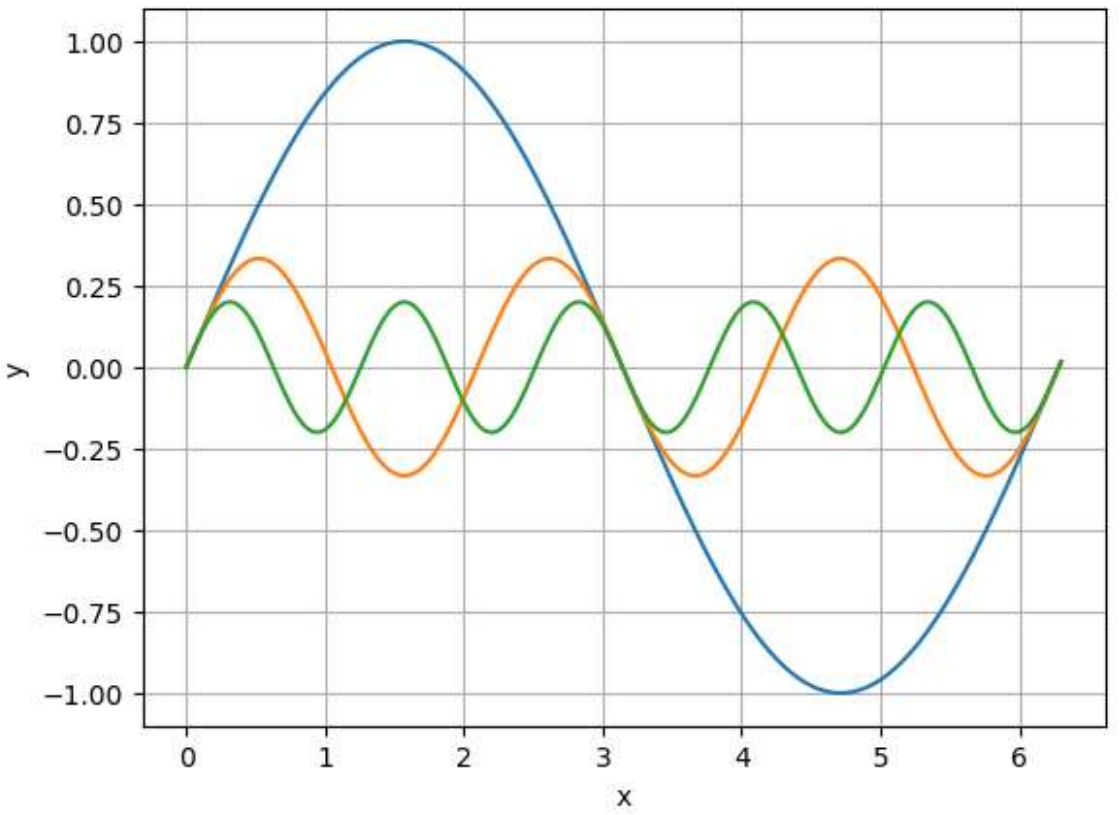
```
y5=np.sin(5*x)/5
y7=np.sin(7*x)/7

y=y1+y3+y5+y7

fig,ax=plt.subplots()
ax.plot(x,y1)
ax.plot(x,y3)
ax.plot(x,y5)
#ax.plot(x,y7)
#ax.plot(x,y)

ax.set_xlabel('x')
ax.set_ylabel('y')
ax.grid(True)

plt.show()
```



3. plot(x,y1),Plot(x,y3), plot(x,y5),plot(x,y7) and plot(x,y)

```
import numpy as np
import matplotlib.pyplot as plt

x=np.linspace(0,6.3,500) # 2pi= 6.283.. so 2pi is aproximately 6.3. 500 p

y1=np.sin(x)
y3=np.sin(3*x)/3 #sin(3*x)means the sine wave oscillates three times fast
```

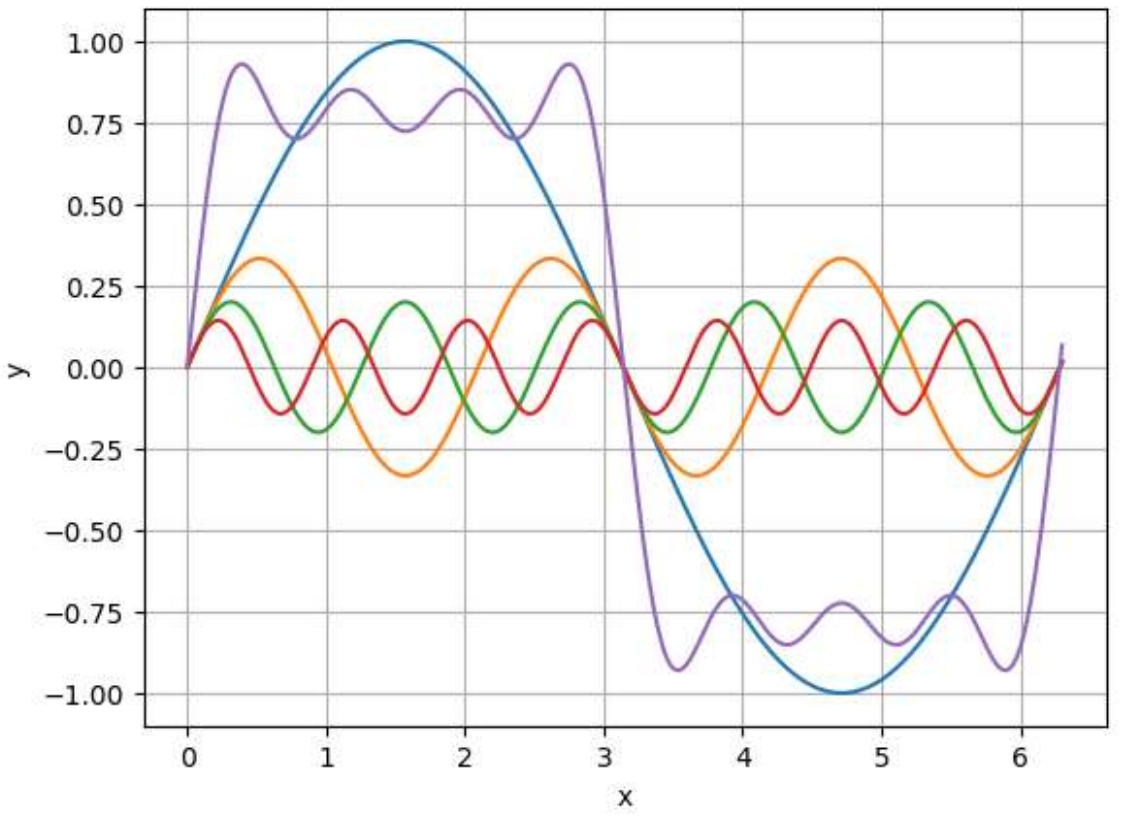
```
y5=np.sin(5*x)/5
y7=np.sin(7*x)/7

y=y1+y3+y5+y7 #superposition of sinusoidal components( complex wave form)

fig,ax=plt.subplots()
ax.plot(x,y1)
ax.plot(x,y3)
ax.plot(x,y5)
ax.plot(x,y7)
ax.plot(x,y)

ax.set_xlabel('x')
ax.set_ylabel('y')
ax.grid(True)

plt.show()
```



linestyle controls the appearance of the line, while color controls its color and lw controls its thickness.

```
import numpy as np
import matplotlib.pyplot as plt

x=np.linspace(0,6.3,500) # 2pi= 6.283.. so 2pi is aproximately 6.3. 500 p

y1=np.sin(x)
```

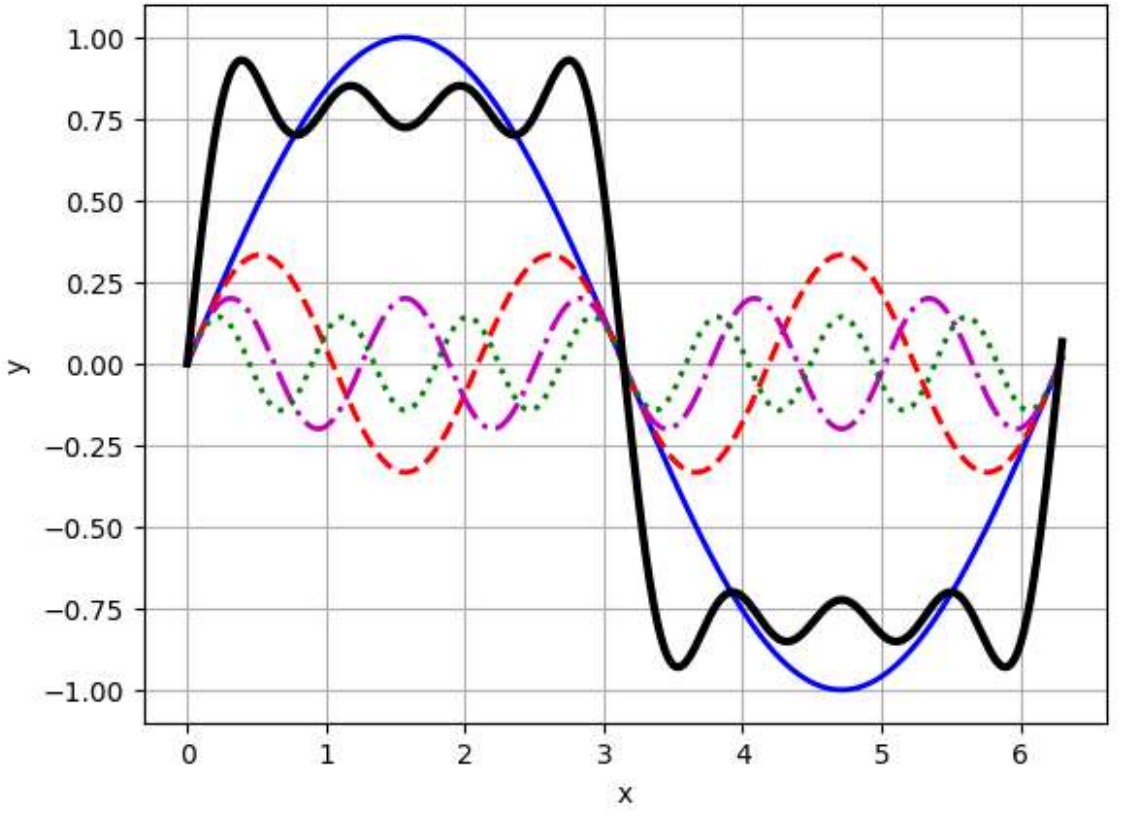
```
y3=np.sin(3*x)/3 #sin(3*x)means the sine wave oscillates three times fast
y5=np.sin(5*x)/5
y7=np.sin(7*x)/7

y=y1+y3+y5+y7 #superposition of sinusoidal components( complex wave form)

fig,ax=plt.subplots()

ax.plot(x,y1,color='b',lw=2,linestyle='-')
ax.plot(x,y3,color='r',lw=2,linestyle='--')
ax.plot(x,y5,color='m',lw=2,linestyle='-.')
ax.plot(x,y7,color='g',lw=2,linestyle=':')
ax.plot(x,y,color='black',lw=3)
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.grid(True)

plt.show()
```



Line style Code Meaning Appearance

'-' Solid ————

'--' Dashed - - - -

'-.' Dash-dot - · - ·

':' Dotted · · · ·

Designing Axes Write a Python program using Matplotlib to plot the relationship between the length of a test bar and the tensile force applied to it.

Length of test bar l in mm:

[0, 0.02, 0.1, 0.2, 1.15, 2.2, 3.25, 4.3, 5.4, 6.4]

Tensile force F in kN:

[5.7, 7.5, 7.2, 7.3, 8.9, 10.4, 11.3, 12, 11.4, 9.3]

```
import matplotlib.pyplot as plt

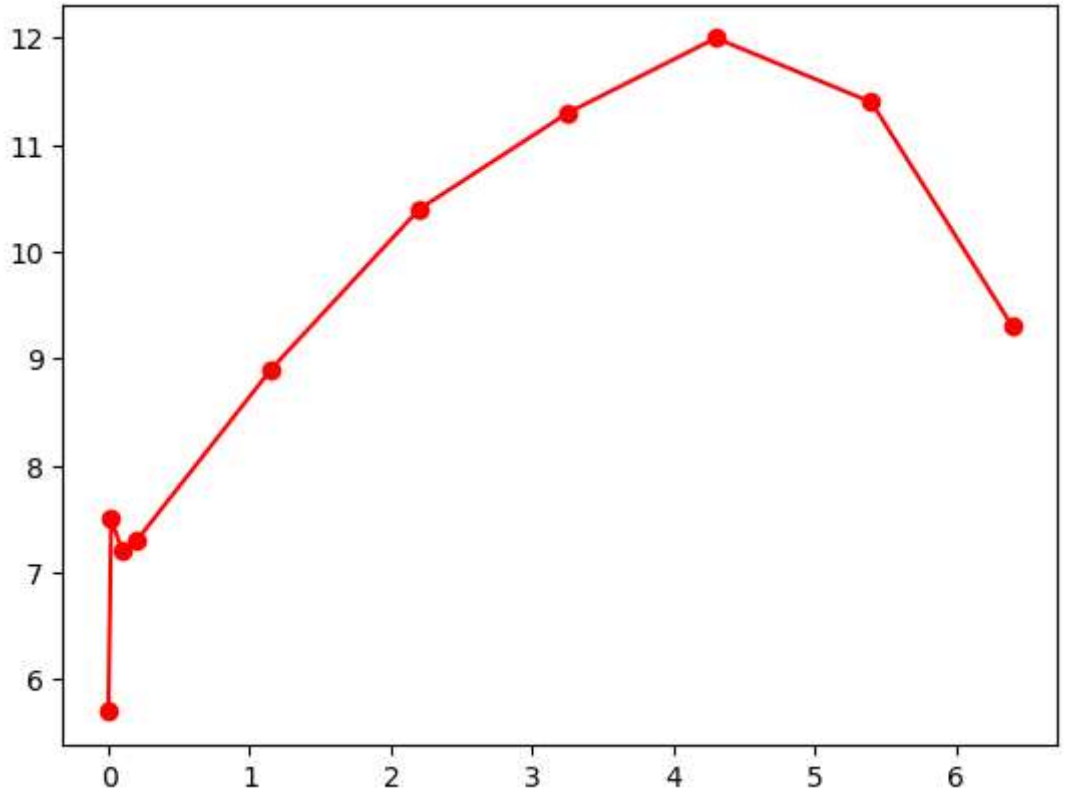
# Test bar length in mm
l = [0, 0.02, 0.1, 0.2, 1.15, 2.2, 3.25, 4.3, 5.4, 6.4]

# Tensile force in kN
F = [5.7, 7.5, 7.2, 7.3, 8.9, 10.4, 11.3, 12, 11.4, 9.3]

# Create Figure and Axes
fig, ax = plt.subplots()

# Plot tensile force against length
ax.plot(l, F, 'ro-') #'ro-' = red + circle marker + solid line
```

```
[<matplotlib.lines.Line2D at 0x7f0b501c2e90>]
```



Let us understand the graph

There are two important things:

Axis

- Horizontal axis ⟶ x-axis
- Vertical axis ⟶ y-axis

Two different controls

We can control:

1. Where the graph starts and ends ⟶ axis limits
2. Where numerical markings appear ⟶ ticks

What are ticks? A tick is a small mark on an axis that indicates a particular numerical value.

a tick is accompanied by a tick label

| ⟶ tick mark

0, 1, 2, 3 ⟶ tick labels

set_xticks() controls tick positions in x axis.

set_xticks([value1,value2,value3,....])

set_yticks() controls tick positions in y axis.

set_yticks([value1,value2,value3,....])

```
import matplotlib.pyplot as plt

# Test bar length in mm
l = [0, 0.02, 0.1, 0.2, 1.15, 2.2, 3.25, 4.3, 5.4, 6.4]

# Tensile force in kN
F = [5.7, 7.5, 7.2, 7.3, 8.9, 10.4, 11.3, 12, 11.4, 9.3]

# Create Figure and Axes
fig, ax = plt.subplots()

# Plot tensile force against length
ax.plot(l, F, 'ro-') #'ro-' = red + circle marker + solid line

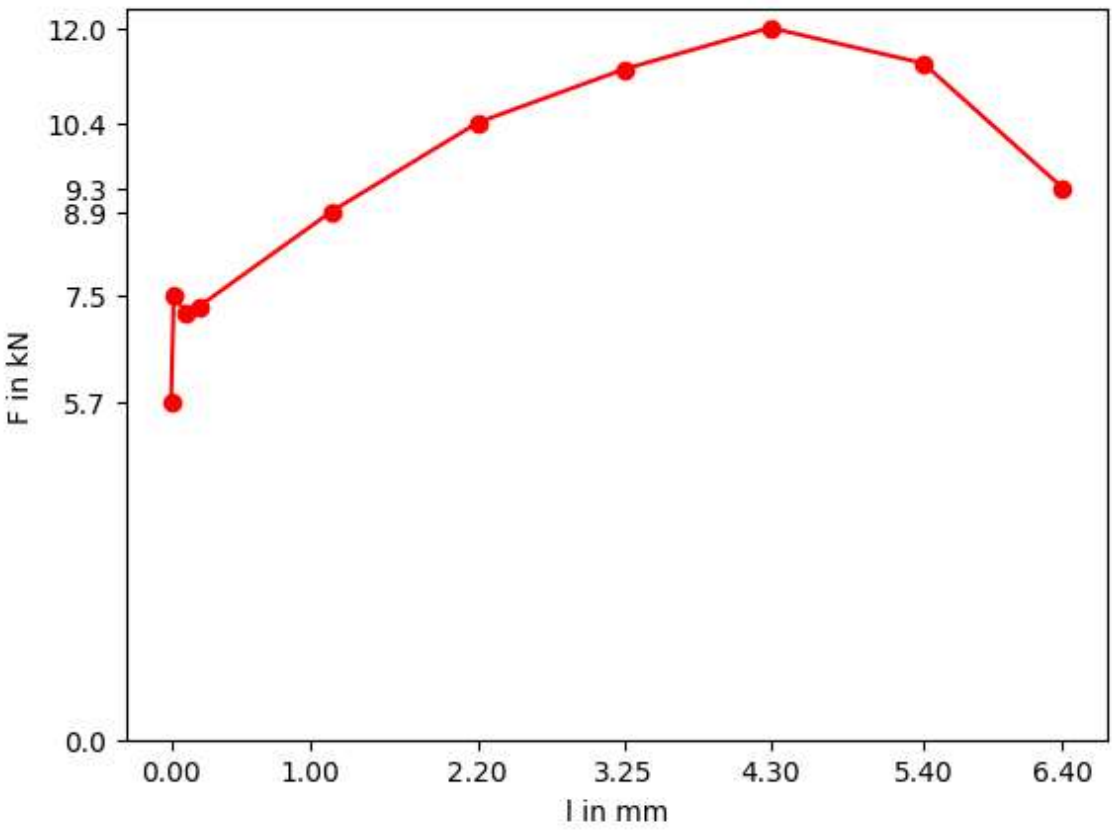
# Set x-axis tick positions
ax.set_xticks([0, 1, 2.2, 3.25, 4.3, 5.4, 6.4])

# Set y-axis tick positions
ax.set_yticks([0, 5.7, 7.5, 8.9, 10.4, 12, 9.3])

# Label the axes
ax.set_xlabel("l in mm")
```

```
ax.set_ylabel("F in kN")

# Display the graph
plt.show()
```



Now what is an axis limit?

An axis limit specifies the visible range of the graph.

ax.axis([-0.5, 7, 5, 13])

x-min = -0.5

x-max = 7

y-min = 5

y-max = 13

So the visible graph area is:

-0.5<= x <=7

5 <= y <=13

```
import matplotlib.pyplot as plt

# Test bar length in mm
l = [0, 0.02, 0.1, 0.2, 1.15, 2.2, 3.25, 4.3, 5.4, 6.4]
```

```
# Tensile force in kN
F = [5.7, 7.5, 7.2, 7.3, 8.9, 10.4, 11.3, 12, 11.4, 9.3]

# Create Figure and Axes
fig, ax = plt.subplots()

# Plot tensile force against length
ax.plot(l, F, 'ro-') #'ro-' = red + circle marker + solid line

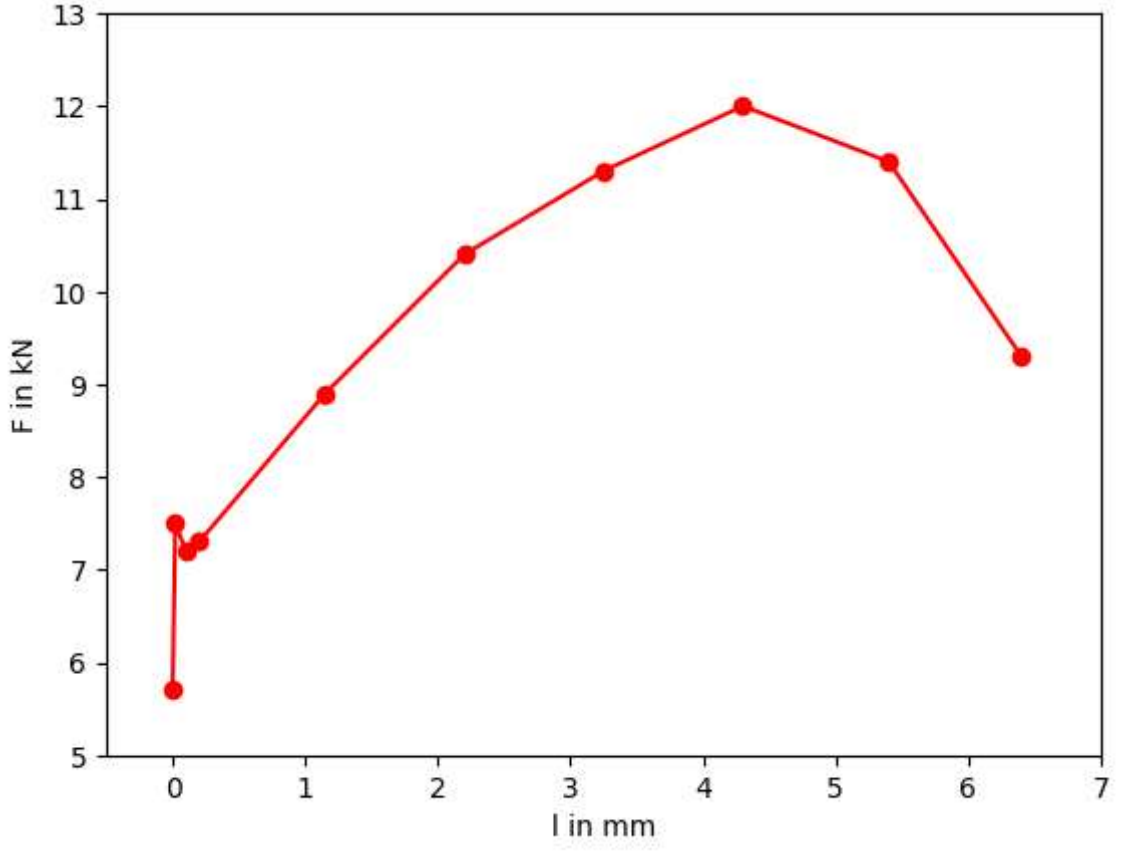
# Set x-axis tick positions
#ax.set_xticks([0, 1, 2.2, 3.25, 4.3, 5.4, 6.4])

# Set y-axis tick positions
#ax.set_yticks([0, 5.7, 7.5, 8.9, 10.4, 12, 9.3])

ax.axis([-0.5,7,5,13])

# Label the axes
ax.set_xlabel("l in mm")
ax.set_ylabel("F in kN")

# Display the graph
plt.show()
```



Axis limit

How much of the coordinate system should I display?

Tick

At which positions should I put numerical markings?